Theme: Open Source & Innovation

- **Problem Statement:** In most of our homes, good food gets pushed to the back of the fridge and rots simply because we forget it's there. On top of that, busy daily schedules make it easy to miss medication times or ignore basic dietary goals. Commercial smart fridges try to solve this, but they cost lakhs of rupees and upload your private home data to company servers. We need an affordable, open-source way to upgrade regular fridges while keeping all personal data completely inside the house.

- **What We Intend to Build:** We are building **FrostMind** —an open-source kit and Python app that turns any regular fridge into a helpful health assistant. Using a cheap camera setup, basic barcode/receipt scanning, and a small local AI model running on Python, it keeps track of what's inside, warns you before food goes bad, and gives quick voice reminders for meds when you open the door. Everything runs locally on the device with zero cloud dependency.

2. Our Build & Task Plan

#	Task	Who owns it	How we do it	Why we split it this way
1	**Gathering foods & receipt photos**	**We Led**	Augmentation	We need to take real photos of our own groceries and local store receipts to handle weird lighting and messy print.
2	**Database & Expiry Logic**	**Shared**	Agency	AI helps write the basic SQLite code, but we decide the table layout, expiry rules, and privacy rules ourselves.
3	**Barcode & Receipt Reader**	**Shared**	Augmentation	AI writes the basic OpenCV and Tesseract scripts, then we clean up the regex to fix bad character reads.
4	**Natural Language Kitchen Queries**	**AI Led**	Automation	🚩 **UNTESTABLE** — Hooking up a lightweight local LLM so users can ask quick questions about what food is left.
5	**Medication Reminder Logic**	**We Led**	Agency	Health alerts and timing triggers cannot be left to AI text generation; we need to write hardcoded logic.
6	**Telegram Alert Bot**	**AI Led**	Automation	Standard Python Telegram bot script to send expiry updates straight to our phones.
7	**Dashboard Interface**	**Shared**	Augmentation	AI builds the initial Streamlit page code; we tweak the UI so it looks good on a fridge touch screen.

🚩**Untestable Flag Note:** Task 4 (Local LLM Queries) is flagged because we can't tell if the AI's answers are fast or accurate enough until we test latency and memory limits directly on our laptop/hardware during the build.

3. Decisions We Will NOT Hand Over to AI

1. **Keeping Data 100% Offline:** We won't let AI decide where data goes. Guaranteeing that camera feeds, receipts, and medication logs stay strictly local requires human code and clear privacy boundaries.
2. **Medication & Health Safety Rules:** We will write and audit every line of code for med reminders and allergen warnings ourselves. Relying on AI text output for health prompts is too risky because AI can hallucinate.
3. **Hardware & Sensor Connections:** Linking physical fridge door triggers and webcam feeds to our Python scripts takes hands-on testing and manual debugging.

Entry 2: your 4D Check table and your delegation map.

Applying the 4D Framework to a Real Task

Real Task for This Week: *Building a Python script to scan receipt text and assign default expiration days.*

- **Delegation:**
  - *Handed to AI:* Writing regex patterns and Python string functions to clean up messy OCR text.
  - *Kept for Us:* Setting default shelf-life rules (e.g., Milk = 7 days, Eggs = 21 days) and deciding what happens when an item isn't recognized.
- **Description (Our Brief to AI):**
  - *Context:* A Python script running locally on an edge device.
  - *Objective:* Turn raw, messy receipt text into a clean item list with expiry dates.
  - *Task:* Write a function parse_receipt(raw_text) using pytesseract and regex.
  - *Constraints:* Offline execution only, outputs valid Python dictionaries, no external API calls.
  - *Output:* {"item_name": "String", "days_valid": Int}
  - *Success Criteria:* Successfully parses at least 8 out of 10 test receipts without throwing errors.
- **Discernment:** We ran the generated code against 10 real receipt photos we took on our phones to check if it actually grabbed item names instead of store addresses or price totals
- **Diligence:** We reviewed the script manually to ensure it handles blank image inputs gracefully and doesn't leak memory during repeated runs.

Our 3 Stop-and-Check Moments

1. **The OCR Scanner Check:** After finishing the camera/receipt parser, we will test 20 real grocery items to make sure it reads names correctly at least 85% of the time before saving them to the database.
2. **The Med Safety Test:** We will simulate 10 different door-open events at different times of day to ensure medication prompts fire when expected without missing alerts or giving wrong messages.
3. **The Offline Audit:** Before recording our final video or demoing to judges, we will turn off Wi-Fi completely and run the app to prove everything runs locally without sending data anywhere.

Entry 3: Research Brief & Evidence Log

1. Our 6-Part Research Brief

- **Context:** We're building **FrostMind** (a cheap, open-source fridge tracker running on Python). To make judges believe our problem actually matters, we need real numbers on how much food regular homes waste, how bad people are at taking daily meds, and why existing smart fridges fail.
- **Objective:** Find solid, trustworthy numbers for food waste, missed meds, and smart fridge pricing/privacy issues.
- **Task:** Look up verified facts on:
  1. How much food an average family throws away every year.
  2. The costs and health issues caused by people forgetting their daily medicines.
  3. How much current smart fridges cost and what privacy risks they have.
- **Constraints:** Stick to real reports like the UN, WHO, or NIH. No random blog posts or made-up statistics.
- **Output:** A short bulleted list of facts with direct links/sources.
- **Success Criteria:** Find at least 3 solid numbers that directly back up our problem statement.

2. What AI Asked Us About Our Brief (And What We Forgot to Decide)

We asked ChatGPT to roast our brief, and it brought up 3 good points we hadn't thought about:

1. *Are we talking about Indian homes or global numbers?*
   - **What we decided:** Use the global UN report for total waste volume, but use average family spending numbers to show the money wasted.
2. *Why connect a fridge to taking medicine?*
   - **What we decided:** Focus on meds people have to take with meals (like diabetes tablets) or store in the cold. Opening the fridge for breakfast/dinner is a natural reminder time.
3. *What privacy issues do people actually care about in smart fridges?*
   - **What we decided:** Point out that smart fridges with cameras upload video feeds to corporate cloud servers and force you to create online accounts.

3. The Raw Research Output We Got

- **Food Waste:** The UN Environment Programme (UNEP) report says households produce 60% of global food waste—about 631 million tonnes a year. An average family throws out around $1,500 (~₹1.2 Lakhs) worth of uneaten food every year.
- **Missed Meds:** Around 50% of people with chronic illnesses miss their scheduled doses. In the US alone, this leads to $100B–$300B in preventable hospital costs every year.
- **Smart Fridges:** Smart fridges like Samsung Family Hub cost $2,000 to $4,000 (₹1.5 Lakhs to ₹3+ Lakhs). They also collect user data and send internal camera footage to external clouds.

4. Double-Checking One Fact (The 3-Step Verification)

1. **The Claim We Picked**: *"Households account for 60% of all food waste, totaling 631 million tonnes annually."*
2. **How We Verified It:**
   - Step 1: Searched for the official UNEP Food Waste Index Report.

- Step 2: Opened the report on the official UN Environment Programme site.
- Step 3: Matched the stat—the report explicitly lists 631 million tonnes generated by households out of 1.05 billion tonnes total (which is exactly 60%).

3. **Source Opened:** UNEP Food Waste Index Report (unep.org).

5. Our Tagged Evidence List

This is our running log of facts, guesses, and assumptions for the pitch:

ID	Fact / Statement	Tag	Source / Justification
EVD-01	Households waste 631 million tonnes of food every year (~60% of all food waste).	**Evidence**	Verified directly from the official UN report.
EVD-02	An average family wastes around $1,500 (~₹1.2 Lakhs) of food yearly.	**Evidence**	From USDA food waste studies.
EVD-03	About 50% of chronic patients don't take their meds on time.	**Evidence**	From NIH / medical research papers.
INF-01	Forgetting stuff sitting at the back of the fridge is a main reason food goes bad.	**Inference**	Makes logical sense when looking at grocery buying habits vs. waste.
INF-02	Showing a med reminder on a screen when someone opens the fridge for a meal will help them remember.	**Inference**	Logical connection: linking mealtime (opening fridge) with taking daily medicine.
HYP-01	People who won't buy a ₹2 Lakh smart fridge will like a cheap ₹4,000 DIY kit that keeps data local.	**Hypothesis**	Our main product idea—we need to test this with real users during feedback.
ASM-01	People will actually take 5 seconds to scan a receipt or barcode after buying groceries.	**Assumption**	We aren't 100% sure people won't be too lazy to scan, so we have to test how smooth we can make this in our app.

Entry 4: Unwind, Reframes, Critic Objections, Three Ranked Assumptions, and Problem Statement v2

1. The Unwind (5 Whys)

1. *Why do we need a smart fridge assistant?* Because people forget what food is inside their fridge until it spoils, and forget to take their daily meds with meals.
2. *Why do they forget what is inside their fridge?* Because leftover items, produce, and opened packages get pushed to the back of dark shelves and become invisible during daily meal prep.
3. *Why don't they just check or log items manually using a phone app?* Because manual tracking takes too much effort. People get lazy typing every item after buying groceries or cooking.

4. *Why do we think a camera + OCR + AI setup solves this friction?* Because scanning receipts or taking a quick shelf snapshot takes under 5 seconds, automating the log-in process.
5. *Why do we assume people will actually use an in-fridge camera or receipt scanner consistently?* We don't know for sure—this is an assumption about human habits that only field user testing can prove.

First Research Question: What is the maximum acceptable setup time (in seconds) or interaction effort that a household member is willing to spend logging groceries before completely abandoning a kitchen tracking app?

2. Reframes

- **Upstream System Lens:** Food waste starts at the grocery store when people over-buy due to lack of a live list. Alerting users while shopping prevents surplus buying before items enter the home.
- **Extreme User Lens:** Elders taking daily medicines or home cooks might not want to navigate a digital UI on a screen. Voice prompts in regional languages or ambient LED colour indicators are far more accessible than a complex dashboard.

3. Critic Objections

- **Critic 1 (Hardware Pragmatist):** Retrofitting a custom camera and wiring inside a moist, cold fridge interior creates major reliability issues. Fogged lenses, battery drain, and messy camera angles will make visual recognition fail constantly.
- **Critic 2 (Behavioural Lazy User Skeptic):** Indian local vendors and small Kirana stores don't issue itemized printed receipts. Expecting users to manually take clean photos of loose vegetables or unbranded bags every single day will fail after week one.

4. Three Ranked Assumptions & 48-Hour Tests

Rank	Assumption	What Breaks If False	48-Hour Test Plan
#1	**Computer vision (OCR/YOLO) can accurately identify items from a messy fridge camera feed or receipt photo.**	**Everything.** If vision parsing fails or misidentifies items constantly, the entire automated inventory system is useless.	**Test:** Take 15 messy photos of actual fridge shelves and Kirana/supermarket receipts using a phone camera under low light. Run them through a basic Python tesseract + OpenCV script to calculate the raw recognition accuracy rate.
#2	**Users will care enough about med reminders on a fridge to link their dosage schedule to meal times.**	**Half the Value Prop.** If people prefer standard phone alarms or pillboxes, the health module becomes useless bloat.	**Test:** Interview 5 family members/relatives taking daily chronic meds (e.g., diabetes, BP) and ask if they've ever missed a dose because it wasn't tied to a physical kitchen habit like getting food.
#3	**A local lightweight LLM (Llama-3-8B/Gemma-2B) can run fast enough on**	**User Experience.** If asking "What can I cook?" takes 30 seconds	**Test:** Deploy a quantized Gemma-2B or Llama-3 model using llama-cpp-python on an unaccelerated CPU/laptop

	basic laptop/edge hardware to answer queries without lag.	to generate an answer, voice interaction becomes unusable.	and time the response latency for 5 sample kitchen queries.

5. Problem Statement Evolution

- **Version 1:** Most urban households throw away good food because items get pushed to the back of the fridge and forgotten. At the same time, balancing daily routines like taking meds on time gets neglected. Commercial smart fridges cost thousands of dollars and upload private data to corporate clouds. We need an open-source, private way to upgrade regular fridges.
- **Version 2 (Build Target):** Urban working families and multi-generational Indian households struggle with high perishable food wastage (losing ₹1,000–₹2,500 monthly in spoiled dairy, cooked leftovers, and produce) because unorganized fridge storage hides items until past expiry. Concurrently, elderly members frequently miss meal-dependent daily prescriptions. Today, households rely on manual memory, sticky notes, or basic phone alarms, which fail due to tracking friction. We are building a low-cost, privacy-first edge kit that passively logs perishables via receipt/photo capture and uses ambient meal-time voice alerts to bridge food visibility and medication compliance.

Entry 5: PMF Hypothesis, ICP, Substitute, Mechanism, Sized Opportunity, Insight Ledger, and Chosen Flow

1. PMF Hypothesis

- **User:** Urban Indian dual-income households and multi-generational families managing daily home cooking.
- **Situation:** When managing weekly perishable groceries and coordinating daily prescription medicine routines for family members.
- **Outcome:** Reduce spoiled food waste by at least 50% and eliminate missed meal-dependent medication doses.
- **Alternative:** Relying on memory, whiteboard notes, phone alarms, or expensive ₹1.5L+ commercial smart fridges.
- **Edge: FrostMind** delivers an open-source, ₹4,000 offline edge-AI retrofit kit that operates entirely on local hardware, protecting household privacy while automating tracking via door-triggered audio-visual prompts.

2. ICP & 5 Named Real Examples

- **ICP Definition:** Urban working professionals or nuclear/joint families in metro/tier-1 cities who spend ₹8,000–₹15,000 monthly on perishable groceries, have at least one family member taking daily chronic prescriptions with meals, and value privacy and cost-effective automation.

- **5 Named Examples:** Uncle Rajesh (54, diabetic), Priya & Amit (dual-income IT couple), Auntie Sunita (manages 5-person joint family kitchen), Dr. Sharma (local general physician), and Rohan (final-year hosteller sharing an apartment).

3. Current Substitute & Real Cost

- **Substitute:** Mental Memory + Manual Sticky Notes + Standard Phone Alarms
- **Real Cost:** ₹1,000 to ₹2,500 monthly in spoiled groceries, extra healthcare costs from missed doses, and daily mental fatigue.

4. Mechanism

FrostMind converts passive kitchen movements into automatic status updates by using door-triggered edge sensors to log item changes and issue meal-time audio-visual prompts at the exact moment of physical interaction, eliminating manual logging effort while keeping all private home data strictly offline.

5. Opportunity Sizing

- **Findable Users:** ~12.5 Million urban middle-to-upper-middle-class households in Tier-1 Indian cities.
- **Realistic Price:** ₹4,000 one-time hardware kit cost (or ₹0 open-source download).
- **Realistic Adoption:** 0.5% market capture = ~62,500 households.
- **Opportunity Size:** $62,500 \ ₹4,000 = {₹25 Crore (~$3 Million USD$) local addressable market.

6. Insight Ledger

Field	Insight Entry #1	Insight Entry #2
Context	Interviewing a 54-year-old chronic diabetes patient about daily prescription routine.	Testing a phone camera OCR scan on a local Kirana receipt with 3 team members.
Observation	Patient silences phone alarms because they ring when he is away from the kitchen, but forgets the pill by the time he finishes serving dinner.	Thermal printed receipts from local stores fade or wrinkle, causing raw OCR to misread item text 35% of the time.
Insight	Alarm timing is useless unless anchored to the physical location and action of eating (opening the fridge/serving food).	Vision/OCR cannot be the sole input; it must be paired with fuzzy string matching against a pre-loaded local food database to correct typos.
Implication	Med reminders must trigger on physical door-open events during designated meal windows rather than static clock timers.	Add a local string-matching algorithm in Python (difflib / Levenshtein distance) to auto-correct messy OCR text before saving.
Tagged Claim	**INF-03** (Habit-anchored physical prompts outperform static digital phone alarms).	**EVD-04** (Unprocessed OCR on thermal receipts has a ~35% character error rate under low light).

7. The One Flow Chosen

1. **User Action:** Paste or scan raw grocery receipt text into the app.

2. **System Processing:** Python backend cleans the text offline using difflib, calculates expiry dates, and saves items to SQLite inventory.db.
3. **User Action:** Open the fridge door (simulated via button) during a meal window (e.g., Breakfast).
4. **System Action: FrostMind** renders a single ambient card showing expiring food items and scheduled mealtime medication reminders.

Entry 6: PROJECT_SPEC.md, Tech Stack with Reasons, Prototype Notes, Test/Stranger Test Notes, and Reusable Workflow

1. PROJECT_SPEC.md Reference

(Full specifications saved in repository as PROJECT_SPEC.md covering requirements REQ-1 through REQ-6 and explicit scope exclusions EXC-1 through EXC-4).

2. Tech Stack & Justifications

- **Frontend UI:** Streamlit (rapid, pure-Python dashboard designed for mounted screens).
- **Backend:** Python 3.11 + OpenCV + tesseract (lightweight offline text/image parsing).
- **Data & Storage:** SQLite3 + difflib (zero-config, local database with fuzzy string matching).
- **Target Hardware:** Raspberry Pi Zero 2 W, reed switch, 4.2" e-ink panel / Laptop CPU.

3. Test & Stranger Test Notes

- **Break Tests:** Tested empty input (""), 5,000-character text blocks, and invalid code inputs. The app handled empty strings with warnings and safely ignored unrecognized text without crashing.
- **Stranger Test Feedback:**
    - Stranger 1 struggled finding where to upload receipts vs where to check door open alerts across tabs.
    - Stranger 2 opened Tab 2 first and was confused by an empty inventory screen.
- **Iterative Change Made:** Merged tab layouts into a single vertical scrolling page and added a "⚡ **Load Sample Receipt**" button for instant one-click testing.

4. Reusable Workflow: "Feature Spec & Verification Workflow"

- **Input:** Raw feature request + PROJECT_SPEC.md constraints.
- **Process:**
    1. Draft minimum Python logic/Streamlit component.
    2. Run 3 breaking edge-case tests (empty, oversized, malformed input).
    3. Review implementation against explicit exclusions in PROJECT_SPEC.md.

- **Verification:** Confirm 100% offline execution and CPU response time under 2.0 seconds.
- **Output:** Clean, spec-compliant code snippet merged into app.py.